Week 6

Product Review and Rating Management System

Scenario:

An e-commerce company wants to collect customer feedback about the products they purchase. Customers can give ratings from 1 to 5 stars and write reviews about product quality and performance. The system stores customer reviews, manages ratings, and analyzes feedback to identify highly rated products and products that need improvement. This helps the company improve product quality and customer satisfaction.

Required Tables and Attributes

Customer Table

- CUSTOMER_ID (Primary Key)
- CUSTOMER_NAME
- EMAIL
- PHONE
- ADDRESS

Product Table

- PRODUCT_ID (Primary Key)
- PRODUCT_NAME
- PRICE
- CATEGORY_ID (Foreign Key)

1. Create Review and Rating Tables:

A Product Review table is created to store customer feedback about purchased products. It contains customer ID, product ID, rating, review comments and review date. Primary key and foreign key constraints are used to maintain data integrity. A CHECK constraint is applied to ensure that ratings are given only between 1 and 5 stars.

```
SQL> CREATE TABLE Product_Review (
  2      REVIEW_ID NUMBER PRIMARY KEY,
  3      CUSTOMER_ID NUMBER,
  4      PRODUCT_ID NUMBER,
  5      RATING NUMBER CHECK (RATING BETWEEN 1 AND 5),
  6      REVIEW_COMMENT VARCHAR2(500),
  7      REVIEW_DATE DATE DEFAULT SYSDATE,
  8      FOREIGN KEY (CUSTOMER_ID) REFERENCES Customer(CUSTOMER_ID),
  9      FOREIGN KEY (PRODUCT_ID) REFERENCES Product(PRODUCT_ID)
 10  );

Table created.
```

Store Customer Feedback and Ratings:

Customer reviews and ratings are inserted into the Product_Review table. Each customer provides feedback about the purchased product along with a rating between 1 and 5 stars.

```
SQL> INSERT INTO Product_Review VALUES (1, 1, 201, 5, 'Excellent product', S
YSDATE);

1 row created.

SQL> INSERT INTO Product_Review VALUES (2, 2, 202, 4, 'Good quality', SYSDAT
E);

1 row created.

SQL> INSERT INTO Product_Review VALUES (3, 3, 203, 3, 'Average product', SYS
DATE);

1 row created.

SQL> INSERT INTO Product_Review VALUES (4, 4, 204, 5, 'Very useful product',
 SYSDATE);

1 row created.

SQL> INSERT INTO Product_Review VALUES (5, 5, 205, 2, 'Poor quality', SYSDAT
E);

1 row created.

SQL> commit;

Commit complete.
```

Display Product Review Details:

The SELECT statement is used to retrieve and display all customer reviews and ratings stored in the Product_Review table.

```
SQL> SELECT * FROM Product_Review;

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
         1           1        201          5
Excellent product
11-NOV-26

         2           2        202          4
Good quality
11-NOV-26

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------

         3           3        203          3
Average product
11-NOV-26

         4           4        204          5
Very useful product
```

```
 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
11-NOV-26

         5           5        205          2
Poor quality
11-NOV-26
```

2. Store Customer Ratings from 1 to 5 Stars:

Ensure this by adding Constraint:

Customer ratings are stored in the Product_Review table. A CHECK constraint is added to the RATING column to ensure that customers can give ratings only from 1 to 5 stars.

```
SQL> ALTER TABLE Product_Review ADD CONSTRAINT chk_rating CHECK (RATING BETW
EEN 1 AND 5);

Table altered.

SQL> INSERT INTO Product_Review VALUES (8, 8, 208, 6, 'Invalid rating', SYSD
ATE);
INSERT INTO Product_Review VALUES (8, 8, 208, 6, 'Invalid rating', SYSDATE)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.CHK_RATING) violated
```

3. Update Review Comments:

The UPDATE statement is used to modify existing customer review comments in the Product_Review table. If a customer wants to change their feedback, the review comment can be updated using REVIEW_ID.

```
SQL> UPDATE Product_Review SET REVIEW_COMMENT = 'Excellent quality and perfo
rmance' WHERE REVIEW_ID = 2;

1 row updated.
```

```
SQL> SELECT * FROM Product_Review;

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
         1           1        201          5
Excellent product
11-NOV-26

         2           2        202          4
Excellent quality and performance
11-NOV-26

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------

         3           3        203          3
Average product
11-NOV-26

         4           4        204          5
Very useful product
```

```
 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
11-NOV-26

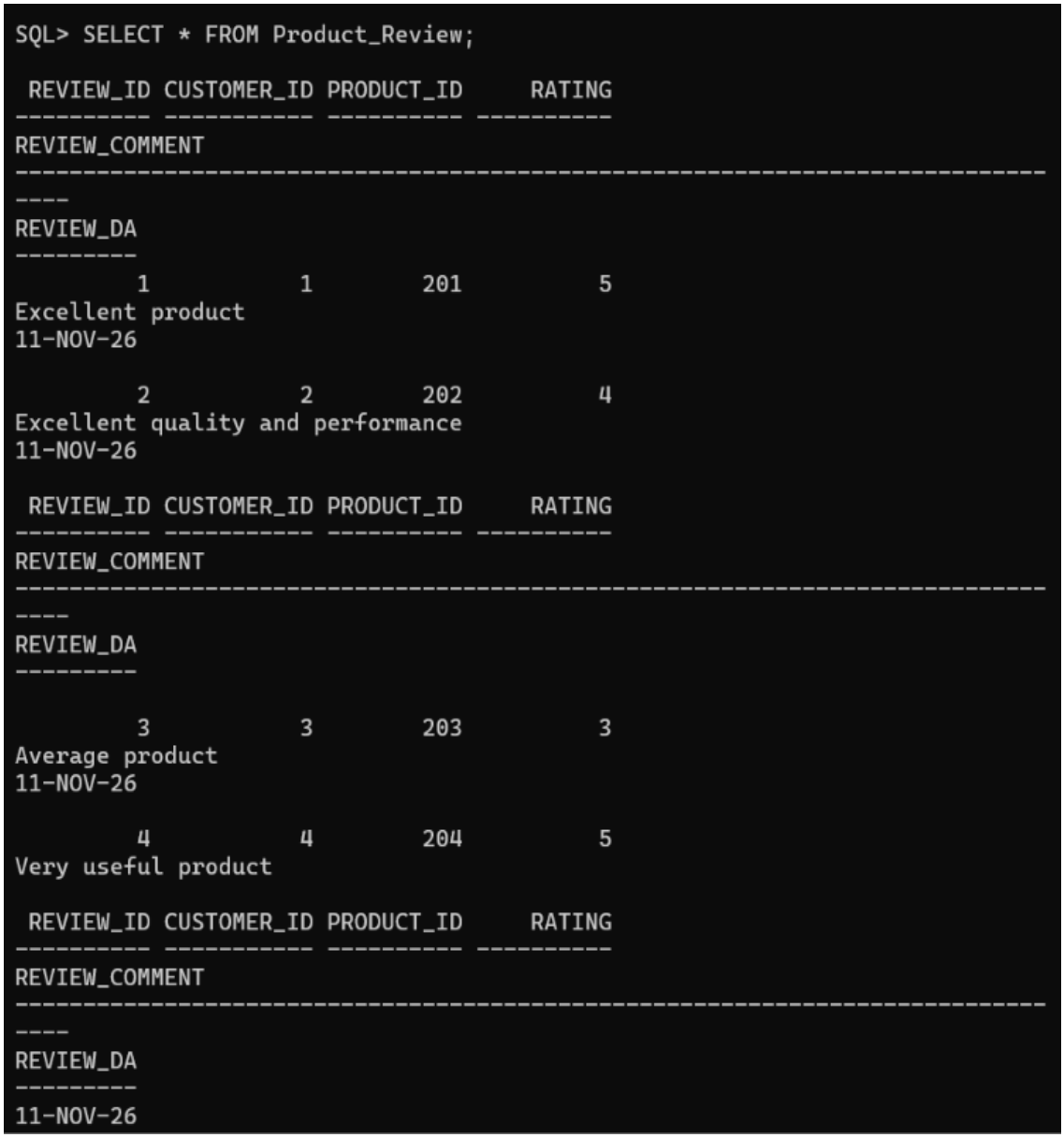
         5           5        205          2
Poor quality
11-NOV-26
```

4. Remove Inappropriate or Invalid Reviews

The DELETE statement is used to remove inappropriate or invalid customer reviews from the Product_Review table. The review is deleted using its unique REVIEW_ID.

```
SQL> DELETE FROM Product_Review WHERE REVIEW_ID = 5;

1 row deleted.
```



5. Retrieve Product Review Details:

1. Display All Reviews for a Product

This query displays all customer reviews and ratings for a specific product using the Product ID.

```
SQL> SELECT * FROM Product_Review WHERE PRODUCT_ID = 201;

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
         1           1        201          5
Excellent product
11-NOV-26
```

2. Display Customer Name with Their Reviews

This query displays the customer name along with their product reviews and ratings by joining the Customer and Product_Review tables.

```
SQL> SELECT  C.CUSTOMER_NAME,R.PRODUCT_ID,R.RATING, R.REVIEW_COMMENT, R.REVI
EW_DATE FROM Customer C JOIN Product_Review R ON C.CUSTOMER_ID = R.CUSTOMER_
ID;

CUSTOMER_NAME                                      PRODUCT_ID     RATING
-------------------------------------------------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
Arun                                                      201          5
Excellent product
11-NOV-26

Priya                                                     202          4
Excellent quality and performance
11-NOV-26

CUSTOMER_NAME                                      PRODUCT_ID     RATING
-------------------------------------------------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------

Kaviya                                                    203          3
Average product
11-NOV-26

Rahul                                                     204          5
Very useful product
```

```
CUSTOMER_NAME                                      PRODUCT_ID     RATING
-------------------------------------------------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
11-NOV-26
```

3. Find Products Having Maximum Reviews

This query is used to find the products having the maximum number of customer reviews.
The JOIN operation combines the Product and Product_Review tables.
The COUNT() function counts the total reviews for each product.
The GROUP BY and HAVING clauses identify the products with the highest number of reviews.

```
SQL> SELECT P.PRODUCT_NAME, COUNT(R.REVIEW_ID) AS TOTAL_REVIEWS FROM Product
 P JOIN Product_Review R ON P.PRODUCT_ID = R.PRODUCT_ID GROUP BY P.PRODUCT_N
AME HAVING COUNT(R.REVIEW_ID) = (SELECT MAX(COUNT(*)) FROM Product_Review GR
OUP BY PRODUCT_ID);

PRODUCT_NAME                                       TOTAL_REVIEWS
-------------------------------------------------- -------------
Mobile                                                         1
Laptop                                                         1
Headphone                                                      1
Keyboard                                                       1
```

4. Display Recent Customer Feedback

This query displays the five most recent customer reviews.
The JOIN operation combines Customer, Product and Product_Review tables.
The ORDER BY clause sorts the reviews by date in descending order.
The FETCH FIRST clause displays only the latest five customer feedback records.

```
SQL> SELECT * FROM (SELECT C.CUSTOMER_NAME, P.PRODUCT_NAME, R.RATING, R.REVI
EW_COMMENT,R.REVIEW_DATE FROM Product_Review R JOIN Customer C ON R.CUSTOMER
_ID = C.CUSTOMER_ID JOIN Product P ON R.PRODUCT_ID = P.PRODUCT_ID ORDER BY R
.REVIEW_DATE DESC) WHERE ROWNUM <= 5;

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME                                       RATING
-------------------------------------------------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
Rahul
Keyboard                                                5
Very useful product
11-NOV-26


CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME                                       RATING
-------------------------------------------------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
Kaviya
Headphone                                               3
Average product
11-NOV-26
```

```
CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME                                       RATING
-------------------------------------------------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
Arun
Laptop                                                  5
Excellent product
11-NOV-26
```

5. Retrieve Reviews with Ratings Above 4:

This query is used to retrieve customer reviews with ratings greater than 4.
The WHERE clause filters the reviews based on the rating value.
It displays only reviews with a 5-star rating.
It helps identify highly rated products based on customer feedback.

```
SQL> SELECT * FROM Product_Review WHERE RATING > 4;

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
         1           1        201          5
Excellent product
11-NOV-26

         4           4        204          5
Very useful product
11-NOV-26

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
```

6. **Calculate Average Rating for Each Product:**
   Use aggregate functions to analyze ratings.

   Tasks:

   1. **Calculate average rating for each product.**

   This query is used to calculate the average rating given by customers for each product.
   The AVG() function calculates the average rating of each product.
   The GROUP BY clause groups the reviews based on product ID.
   It helps the company understand customer satisfaction for each product.

```
SQL> SELECT PRODUCT_ID, AVG(RATING) AS AVERAGE_RATING FROM Product_Review GR
OUP BY PRODUCT_ID;

PRODUCT_ID AVERAGE_RATING
---------- --------------
       201              5
       202              4
       203              3
       204              5
```

   2. **Count the number of reviews per product.**
      This query is used to count the total number of customer reviews for each product.
      The COUNT() function calculates the number of reviews.

The GROUP BY clause groups the reviews based on product ID.
It helps identify which products receive more customer feedback.

```
SQL> SELECT PRODUCT_ID, COUNT(REVIEW_ID) AS TOTAL_REVIEWS FROM Product_Revie
w GROUP BY PRODUCT_ID;

PRODUCT_ID TOTAL_REVIEWS
---------- -------------
       201             1
       202             1
       203             1
       204             1
```

3. Find highest-rated products.

This query is used to identify the products with the highest average customer ratings.
The AVG() function calculates the average rating for each product.
The GROUP BY clause groups the reviews based on product ID.
The HAVING clause filters and displays only the products with the highest average rating.

```
SQL> SELECT PRODUCT_ID, AVG(RATING) AS AVERAGE_RATING FROM Product_Review GR
OUP BY PRODUCT_ID HAVING AVG(RATING) = (SELECT MAX(AVG(RATING)) FROM Product
_Review GROUP BY PRODUCT_ID);

PRODUCT_ID AVERAGE_RATING
---------- --------------
       201              5
       204              5
```

4. Identify Products with Average Rating Above 4

This query is used to identify products with an average rating greater than 4.
The AVG() function calculates the average rating for each product.
The GROUP BY clause groups the reviews based on product ID.
The HAVING clause filters and displays products with an average rating above 4.

```
SQL> SELECT PRODUCT_ID, AVG(RATING) AS AVERAGE_RATING FROM Product_Review GR
OUP BY PRODUCT_ID HAVING AVG(RATING) > 4;

PRODUCT_ID AVERAGE_RATING
---------- --------------
       201              5
       204              5
```

5. Identify Highly Rated Products
Generate product rating reports:

Report 1: Product Rating Analysis:

This report displays the rating details of each product based on customer reviews.
The COUNT() function calculates the total number of reviews for each product.
The AVG() function calculates the average rating given by customers.
The JOIN operation combines the Product and Product_Review tables to display product names.

```
SQL> SELECT P.PRODUCT_NAME, COUNT(R.REVIEW_ID) AS TOTAL_REVIEWS, AVG(R.RATIN
G) AS AVERAGE_RATING FROM Product P JOIN Product_Review R ON P.PRODUCT_ID =
R.PRODUCT_ID GROUP BY P.PRODUCT_NAME;

PRODUCT_NAME                                       TOTAL_REVIEWS AVERAGE_RAT
ING
-------------------------------------------------- ------------- -----------
---
Mobile                                                         1
  4
Laptop                                                         1
  5
Headphone                                                      1
  3
Keyboard                                                       1
  5
```

Report 2: Customer Feedback Analysis:

1. Most Reviewed Products

:

This query is used to identify the products that receive the highest number of customer reviews.
The COUNT() function calculates the total number of reviews for each product.
The GROUP BY clause groups the reviews based on product name.
The ORDER BY clause arranges the products from highest to lowest review count.

```
SQL> SELECT P.PRODUCT_NAME, COUNT(R.REVIEW_ID) AS TOTAL_REVIEWS FROM Product
 P JOIN Product_Review R ON P.PRODUCT_ID = R.PRODUCT_ID GROUP BY P.PRODUCT_N
AME ORDER BY TOTAL_REVIEWS DESC;

PRODUCT_NAME                                       TOTAL_REVIEWS
-------------------------------------------------- -------------
Mobile                                                         1
Keyboard                                                       1
Headphone                                                      1
Laptop                                                         1
```

2. Highly Rated Products:

This query is used to identify products with the highest average customer ratings.
The AVG() function calculates the average rating for each product.
The GROUP BY clause groups the reviews based on product name.
The ORDER BY clause arranges products from highest to lowest average rating.

```
SQL> SELECT P.PRODUCT_NAME, AVG(R.RATING) AS AVERAGE_RATING FROM Product P J
OIN Product_Review R ON P.PRODUCT_ID = R.PRODUCT_ID GROUP BY P.PRODUCT_NAME
ORDER BY AVERAGE_RATING DESC;

PRODUCT_NAME                                       AVERAGE_RATING
-------------------------------------------------- --------------
Laptop                                                          5
Keyboard                                                        5
Mobile                                                          4
Headphone                                                       3
```

3. Products Requiring Improvement

This query is used to identify products that need improvement based on customer ratings.
The AVG() function calculates the average rating for each product.
The GROUP BY clause groups the reviews based on product name.
The HAVING clause displays products with an average rating below 3.

```
SQL> SELECT P.PRODUCT_NAME, AVG(R.RATING) AS AVERAGE_RATING FROM Product P J
OIN Product_Review R ON P.PRODUCT_ID = R.PRODUCT_ID GROUP BY P.PRODUCT_NAME
HAVING AVG(R.RATING) < 3;

PRODUCT_NAME                                       AVERAGE_RATING
-------------------------------------------------- --------------
Mouse                                                           2
```

Report 3: Rating Distribution:

1. Number of 5-Star Ratings

This query is used to count the total number of 5-star ratings given by customers.
The COUNT() function calculates the number of 5-star reviews.
The WHERE clause filters the reviews with a rating of 5.
It helps identify the number of highly satisfied customers.

```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATINGS FROM Product_Review WHERE RATING =
 5;

FIVE_STAR_RATINGS
-----------------
                2
```

2. Number of 4-Star Ratings

This query is used to count the total number of 4-star ratings given by customers.
The COUNT() function calculates the number of 4-star reviews.
The WHERE clause filters the reviews with a rating of 4.
It helps identify the number of customers who are satisfied with the products.

```
SQL> SELECT COUNT(*) AS FOUR_STAR_RATINGS FROM Product_Review WHERE RATING =
 4;

FOUR_STAR_RATINGS
-----------------
                1
```

3. Number of Low-Rated Products

This query is used to count the total number of products with low customer ratings.
The AVG() function calculates the average rating for each product.
The GROUP BY clause groups reviews based on product ID.
The HAVING clause identifies products with an average rating below 3.

```
SQL> SELECT COUNT(*) AS LOW_RATED_PRODUCTS FROM (SELECT PRODUCT_ID FROM Prod
uct_Review GROUP BY PRODUCT_ID HAVING AVG(RATING) < 3);

LOW_RATED_PRODUCTS
------------------
                1
```

4. **Complete Rating Distribution**:

This query is used to display the complete distribution of customer ratings.
The COUNT() function calculates the number of reviews for each rating.
The GROUP BY clause groups the reviews based on rating values.
It helps the company understand customer satisfaction levels.

```
SQL> SELECT RATING, COUNT(*) AS TOTAL_REVIEWS FROM Product_Review GROUP BY R
ATING ORDER BY RATING DESC;

    RATING TOTAL_REVIEWS
---------- -------------
         5             2
         4             1
         3             1
         2             1
```

Conclusion

Thus, the Product Review and Rating Management System was successfully implemented using SQL queries. Customer reviews and ratings were stored, updated, deleted and retrieved. Aggregate functions such as COUNT() and AVG() were used to analyze product ratings. Different reports were generated to identify highly rated products, most reviewed products and products requiring improvement. This system helps the company understand customer feedback and improve product quality and customer satisfaction.